



Text Generation Project

30.05.2025

RAJ PRATEEK PRAJAPATI

Text Generation Project - Word-Level LSTM (Google Colab)

Summary

This project implements a word-level text generation model using an LSTM neural network trained on Mary Shelley's Frankenstein. Initially, the project was attempted in VS Code using Python 3.13.3. However, since TensorFlow is not yet compatible with Python 3.13.3, there were two options: either install Python 3.10 to support TensorFlow locally or use a cloud-based environment. I chose to use Google Colab, which offers free GPU access and comes with TensorFlow pre-installed.

The model was trained using sequences of words (not characters) to generate coherent text. A cleaned and tokenized version of the text was used to create input sequences, which were then padded and one-hot encoded for model training. The trained model was able to generate sequential words that resemble the structure and vocabulary of the original text.

Project Setup & Execution in Google Colab

Step 1: Upload the Dataset File

```
from google.colab import files
uploaded = files.upload()

file_content = open("frankenstein-2.txt", encoding="utf-8").read()
print(file_content[:500]) # Preview text
```

Step 2: Install Required Libraries

```
!pip install tensorflow
import nltk
nltk.download('punkt')
```

Step 3: Preprocess the Text (Word Tokenization)

```
from nltk.tokenize import word_tokenize

# Tokenize and reformat for Keras
tokens = word_tokenize(file_content.lower())
processed_text = " ".join(tokens)
```

Step 4: Prepare Input Sequences

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

# Tokenization and n-gram sequence creation
tokenizer = Tokenizer()
tokenizer.fit_on_texts([processed_text])
total_words = len(tokenizer.word_index) + 1

input_sequences = []
for line in processed_text.split("\n"):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        input_sequences.append(token_list[i+1])

# Pad and split sequences
max_seq_len = max([len(seq) for seq in input_sequences])
input_sequences = pad_sequences(input_sequences, maxlen=max_seq_len,
padding='pre')
X = input_sequences[:, :-1]
y = to_categorical(input_sequences[:, -1], num_classes=total_words)
```

Step 5: Build and Train the Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential()
model.add(Embedding(total_words, 100, input_length=max_seq_len-1))
model.add(LSTM(150))
model.add(Dense(total_words, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=50, verbose=1)
```

Step 6: Generate Text

```
seed_text = "life is full"
next_words = 20

for _ in range(next_words):
```

```
token_list = tokenizer.texts_to_sequences([seed_text])[0]
token_list = pad_sequences([token_list], maxlen=max_seq_len-1, padding='pre')
predicted = model.predict(token_list, verbose=0)
predicted_index = np.argmax(predicted)
for word, index in tokenizer.word_index.items():
    if index == predicted_index:
        seed_text += " " + word
        break

print("\nGenerated Text:\n", seed_text)
```

Notes

- This project was completed on Google Colab due to TensorFlow's lack of support for Python 3.13.3 in VS Code.
 - The original idea was attempted in VS Code, but Python version conflicts led to the use of Colab.
 - Using character-level models initially led to repeated output (e.g., "eeee..."). Switching to word-level solved this.
 - Training for more epochs will significantly improve the quality of generated text.
-